GÜMÜŞHANE ÜNİVERSİTESİ
İKTİSADİ ve İDARİ BİLİMLER FAKÜLTESİ
YÖNETİM BİLİŞİM SİTEMLERİ BÖLÜMÜ

YBS 207 Algoritma ve Programlamaya Giriş

Tuple Data Type

Öğr. Gör. Dr. Baki GÖKGÖZ

Giriş

- **Tuple Temelleri**
  - Tuple, listeye benzer şekilde, sıralı bir değer koleksiyonudur.

```
my_tuple = (1, 2, 3, 4)
#Output: (1, 2, 3, 4)
```

  - Tupledeki değerler sıralarını korur ve indeksleri aracılığıyla erişilebilir.

```
print(my_tuple[2])
# Output: 3
```

  - Tuplellerin sözdizimi parantez içerir, örneğin,

```
my_tuple = (1, 2, 3)
```

- **Değişmezlik (Immutability)**
  - Demetler (tuple) değiştirilemezdir:
    - oluşturulduktan sonra elemanları değiştirilemez, eklenemez veya silinemez.

```
my_tuple = (1, 2, 3, 4)
my_tuple[1]=52
```



TypeError: 'tuple' object does not support item assignment

  - Bu değişmezlik özelliği ile **demetler** değiştirilebilir olan **listelerden** ayrılır.
  - Verilerin program boyunca sabit kalması sağlanmak istediğinde demetler kullanılır.

Giriş

- **Demetlerin Kullanım Alanları**
  - Sabit bir değer kümesine ihtiyaç duyulduğunda.
    - Örnek:
      - sabit koordinatları `(x, y)` temsil etmek veya sabit yapılandırma verilerini saklamak için.
  - Demetler, listelerin aksine **hashlenebilir** oldukları için sözlüklerde anahtar olarak da kullanılabilirler.
- **Listelerle Benzerlik**
  - Demetler ve listeler, indeksleme, dilimleme ve yineleme gibi birçok özelliği paylaşır.
  - **Listeleri anlamak, demetleri hızlı bir şekilde öğrenmek için sağlam bir temel sağlar.**

Tuple

- **Demet (Tuple) nedir?**
  - Demet, liste gibi bir **Python** veri yapısıdır.
  - Sıralıdır, indekslemeyi destekler, yinelenen öğelere izin verir ve heterojen türler (tamsayılar, ondalık sayılar, dizeler, boole değerleri, karmaşık sayılar) içerebilir.
  - Listelerin aksine, demetler değiştirilemezdir, yani oluşturulduktan sonra içerikleri değiştirilemez.
- **Listeler ve Demetler Arasındaki Farklar**
  - Listeler köşeli parantez `` `[]` `` kullanırken, demetler yuvarlak parantez `` `()` `` kullanır.
  - Listeler değiştirilebilir (elemanlar değiştirilebilir, eklenebilir veya silinebilir).
  - Demetler değiştirilemez (oluşturulduktan sonra hiçbir değişiklik yapılamaz).
  - Demetler, yeniden boyutlandırma veya değiştirme yükü olmadığı için daha hafiftir.
  - Demetler, özellikle veriler salt okunur olduğunda:
    - bir fonksiyon tarafından döndürüldüğünde
    - verilerin yanlışlıkla **değiştirilmemesi** sağlanmak istendiğinde kullanışlıdır.

Tuple

- **Demet Oluşturma Yöntemleri**
  - Virgülle ayrılmış değerlerle parantez kullanımı:

```
t1 = (1, 2, 3, 4, 5)
```

  - `tuple()` kurucusunu yinelenebilir bir nesneyle `(list, string, range)` kullanımı:

```
t2 = tuple([1, 2, 3])
t3 = tuple('python')
t4 = tuple(range(1, 5))
print(t2)
print(t3)
print(t4)
```

  - Boş bir demet oluşturma:

```
t_empty = ()
```

  - Tek elemanlı bir demet oluşturmak için sonuna virgül konulması gerekir:

```
t_single = (5,)
```

  - Virgül olmadan, `` `(5)` `` bir tamsayı olarak yorumlanır.
  - Parantez kullanmadan birden fazla değer atayarak demet oluşturma:

```
t5 = 10, 11, 12, 13, 14
```

Tuple

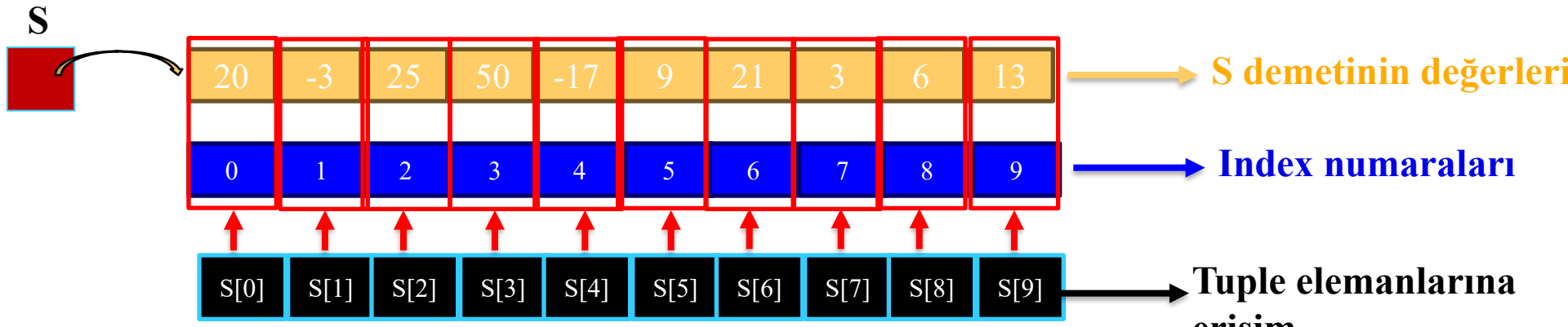
- **Demet Özellikleri ve Davranışı**
  - **Sıralı:** Elemanların indeksleri 0'dan başlar.
  - İndeksleme **pozitif** ve **negatif** indeksleri destekler.
  - **Değiştirilemez:**
    - Yeni bir değer atama veya elemanları değiştirme girişimleri `TypeError` hatası verir.
  - Gezinme, listelere benzer döngüler kullanılarak yapılır,
    - Örnek:

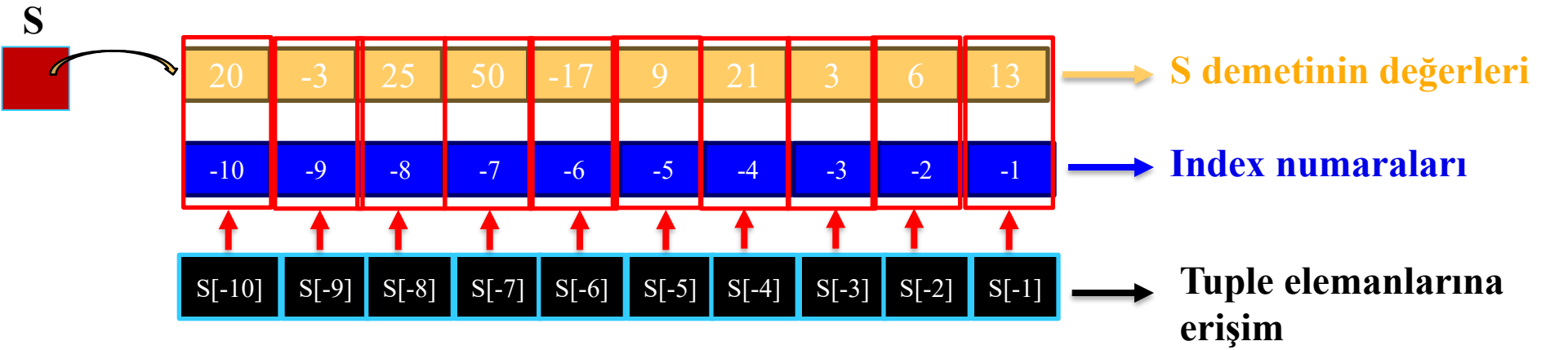
```
tuple = (1, 2, 3, 4, 5)
for x in tuple:
    print(x)
```

- **Demet Gösterimi**
  - Demetler yazdırıldığında parantez içinde gösterilen öğeleri içerir.
  - Dahili olarak, bir değişken demet nesnesine referans verir ve öğeler ayrı, değiştirilemez nesneler olarak bulunur.

Tuple

- **Demet Gösterimi**
  - Demetler yazdırıldığında parantez içinde gösterilen öğeleri içerir.
  - Dahili olarak, bir değişken demet nesnesine referans verir ve öğeler ayrı, değiştirilemez nesneler olarak bulunur.





Tuple Oluşturma Yapıları ve Metotları

- **Unpacking kullanarak Generator'dan Tuple oluşturma**
  - Bir generator ifadesini tuple'a dönüştürmek için, parantezler içinde unpacking (*) kullanılır:
  - **`T = (* (expression for item in iterable), )`**
  - Örnek:

```
T1 = (*(x for x in range(1, 5)), )
print(T1)
# Output: (1, 2, 3, 4)
```

  - 1'den 4'e kadar olan sayılardan, açma işlemi kullanarak bir demet **(tuple)** oluşturur.
  - Generator ifadesinden unpacking kullanmadan tuple oluşturma yöntemi.
  - Örnek: **`tuple()`** fonksiyonunu kullanarak **tuple** oluşturma

```
T2 = tuple(x for x in range(1, 5))
print(T2)
# Output: (1, 2, 3, 4)
```

  - Örnek: **`tuple()`** kullanarak **tuple** oluşturma sırasında kare değerleri hesaplar.

```
T3 = tuple(x**2 for x in range(1, 5))
print(T3)
# Output: (1, 4, 9, 16)
```

Tuple Oluşturma Yapıları ve Metotları

- Örnek: Bir stringden küçük harf karakterlerinden oluşan **tuple** oluşturma

```
T4 = (*(x.lower() for x in 'PyThoN'), )
print(T4)
# Output: ('p', 'y', 't', 'h', 'o', 'n')
```

- 1'den 4'e kadar olan sayılardan, açma işlemi kullanarak bir demet **(tuple)** oluşturur.
- Generator ifadesinden unpacking kullanmadan tuple oluşturma yöntemi.
- Örnek: `tuple()` fonksiyonunu kullanarak **tuple** oluşturma

```
T2 = tuple(x for x in range(1, 5))
print(T2)
# Output: (1, 2, 3, 4)
```

- Örnek: tuple() kullanarak tuple oluşturma sırasında kare değerleri hesaplar.

```
T3 = tuple(x**2 for x in range(1, 5))
print(T3)
# Output: (1, 4, 9, 16)
```

- Dizideki tüm karakterleri küçük harfe dönüştürür ve bunları bir demet (tuple) halinde paketler.

Tuple Oluşturma Yapıları ve Metotları

- Örnek: Rakam dizisinden tamsayılar demeti oluşturma

```
T = (*(int(x) for x in '12345'), )
print(T)
# Output: (1, 2, 3, 4, 5)
```

- Her karakter rakamını tam sayıya dönüştürür ve bir demet oluşturur.
- Örnek: Koşullu demet (sadece alfabetik karakterler)

```
T = (*(x for x in 'ab*cd7e' if x.isalpha()), )
print(T)
# Output: ('a', 'b', 'c', 'd', 'e')
```

- Tuple oluşturmadan önce yalnızca alfabetik karakterleri filtreler.

Indexing & Slicing - Tuple

- **Tuple Slicing Syntax:**
  - Bir tuple içinden bir alt dizi (dilim) çıkarma işlemidir.
    - Genel sözdizimi: `tuple[start:stop:step]`
    - start: Başlangıç indeksi (dahil).
    - stop: Bitiş indeksi (hariç).
    - step: Elemanlar arasındaki atlama miktarı (varsayılan değer 1'dir).

range [start: stop: step]

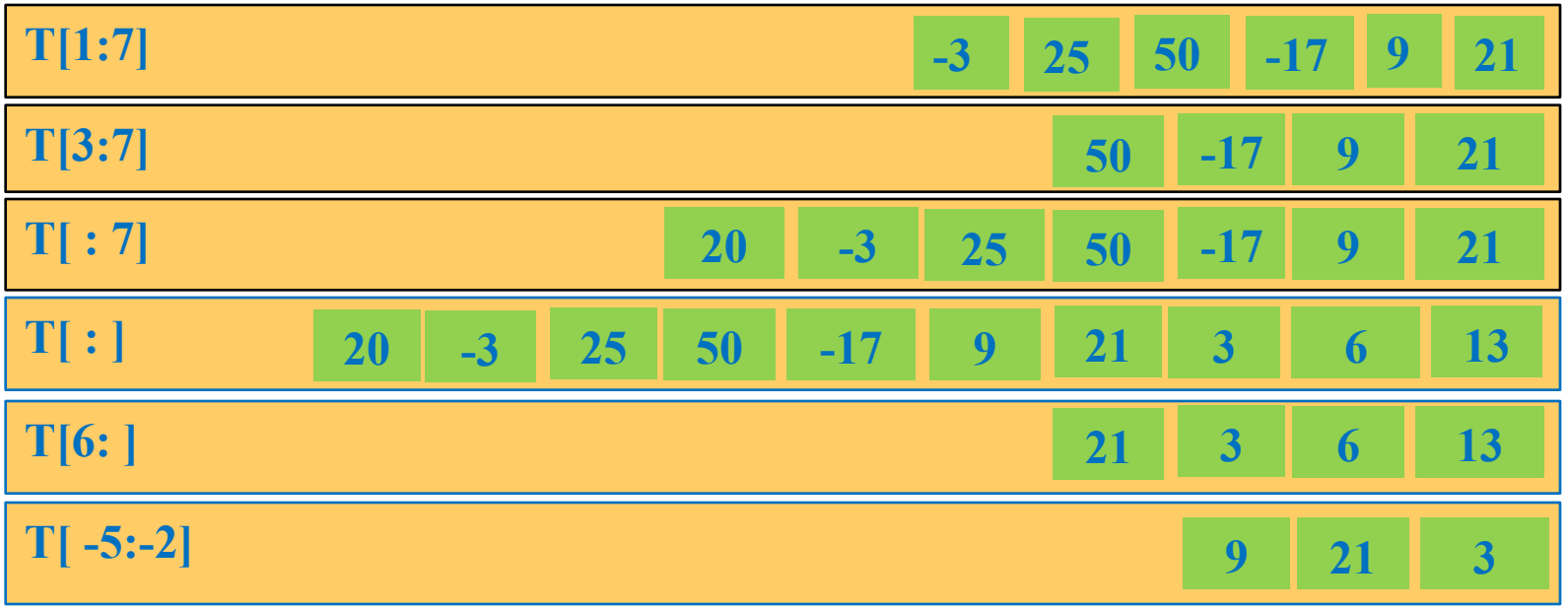
  - Parametreler:
    - Üç parametrenin tamamı isteğe bağlıdır:
    - Start: belirtilmezse varsayılan olarak 0 kabul edilir.
    - Stop: belirtilmezse varsayılan olarak tuple'ın sonuna kadar gider.
    - Step: belirtilmezse varsayılan değeri 1'dir (ileri yönde ilerleme).
    - Pozitif indeksler: ileri yönde dilimleme yapar.
    - Negatif indeksler: start ve stop için kullanılarak sondan itibaren dilimleme yapılmasını sağlar.
    - Negatif step: dilimleme yönünü tersine çevirir ve geriye doğru ilerlemeyi mümkün kılar.
    - step negatif olduğunda: indeks mantığına göre start, stop değerinden büyük olmalıdır.

Indexing & Slicing - Tuple

- **Tuple Slicing Syntax:**

  – Python kodları:

```
T = [20, -3, 25, 50, -17, 9, 21, 3, 6, 13]
```



Indexing & Slicing - Tuple

- **Demet Birleştirme**
  - Operatör: '+'
  - İkinci demetin tüm elemanlarını birinciye ekleyerek iki demeti birleştirir.
  - Yeni bir demet oluşturur; orijinal demetler değişmeden kalır.
  - Örnek:

```
t1 = (1, 2, 3)
t2 = (8, 9, 10)
t3 = t1 + t2
print(t3)
# Output: (1, 2, 3, 8, 9, 10)
```

  - t1 ve t2'yi birleştirerek yeni bir t3 demeti oluşturur.

Indexing & Slicing - Tuple

- **Demet Tekrarlama**
  - Operatör: '*'
  - Tüm demeti belirtilen sayıda tekrarlar.
  - Tekrarlanan elemanlarla yeni bir demet oluşturur.
  - Örnek:

```
t1 = (1, 2, 3)
t2 = t1 * 3
print(t2)
# Output: (1, 2, 3, 1, 2, 3, 1, 2, 3)
```

- **Tupleda Bulunma İşlemleri**
  - **in:** Bir değerin bir demette olup olmadığını kontrol eder, `True/False` döndürür.
  - **not in:** Bir değerin bir demette olmadığını kontrol eder, **True/False** döndürür.

```
t1 = (1, 2, 3, 4, 5)
print(3 in t1)
# Output: True
print(10 not in t1)
# Output: True
```

  - Bir demette elemanların olup olmadığının kontrol edilmesi.